

Socially Responsible Computing Final Documentation



Illmi Yoon, Ph.D Department of Computer Science CSC 630 - Spring
2024 Final Documentation

By: Maya Bal, Aketzali Zeledon, Luis Carrillo, and Rafael Leitao

Table of Contents

Table of Contents	2
Abstract	3
Introduction	3
Background	3
Related Work	4
Motivation	4
Design	6
Milestone 1 Mockup:	6
Milestone 2 Mockup:	7
Implementation/Project Milestones	8
Milestone 1:	8
Milestone 2:	11
Results	13
Without Failed Pixel Overlapping Implementations:	13
With Failed Pixel Overlapping Implementations:	15
Code and Resource Organization	16
Challenges/Problem Solving	16
My Contribution(Do on our own)	17
Learning Reflection(Do on our own)	17

Abstract

Basing our project on the socially responsible aspect of computing, we made our project to include computer graphics being used to design a hearing aid for the person's ear. To be more specific, we wanted to target the design of the hearing aid for infants and young children. In our project we started off with a premade ear mesh and we accommodated the hearing aid to fit the ear as best as possible. The objective of the project is to be able to fit the hearing aid mesh perfectly onto the ear mesh of the person. In theory it would be able to show you where the hearing aid and the ear are touching to make sure it fits correctly.

Introduction

Background

Hearing aids are crucial devices that enable individuals with hearing impairments to engage more fully with their environment by amplifying sounds to accessible levels. Traditionally, these devices include components such as a microphone, amplifier, receiver (speaker), battery, and a processor that manages sound capture and output. In recent years, the development of digital hearing aids has incorporated advanced signal processing technologies to improve the clarity and quality of sound.

Hearing aids are electronic devices designed to improve hearing by making sound more accessible to people with hearing loss. Most hearing aids consist of several key components:

Microphone: Captures sound from the environment.

Amplifier: Increases the volume of the captured sound.

Receiver (Speaker): Delivers the amplified sound into the ear.

Battery: Powers the device.

Processor: In digital hearing aids, a processor converts sound into digital signals, processes it to improve clarity, and then converts it back into sound.

Our interest in this aspect of computer graphics came from looking into the health field and wanting to inform people about devices that people might not be too familiar with. We browsed until we found an item that was of interest to us, we decided on the hearing aid. Keeping in mind that we wanted to use what we learned in this class, the proposal of this project was made. Informing and letting the user play around with the hearing aid was our goal.

Related Work

Prior studies and projects have demonstrated the utility of 3D modeling and visualization in various medical fields, including prosthetics, personalized medicine, and educational tools for healthcare professionals and patients alike. Notably, the customization of medical devices using 3D technology has shown significant promise, especially in creating devices tailored to the unique anatomical features of individuals.

Motivation

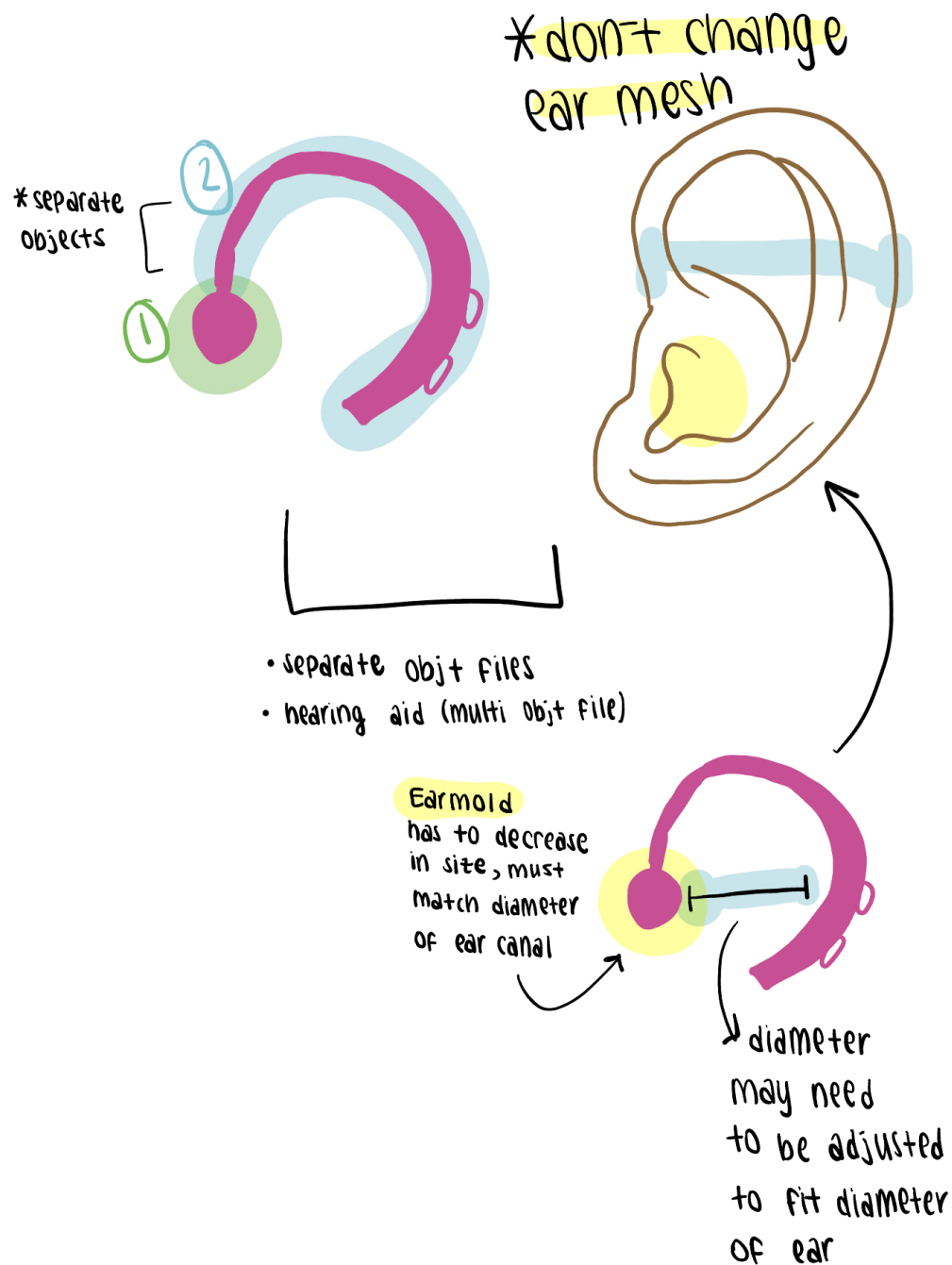
Our project was inspired by the intersection of computer graphics and healthcare, specifically focusing on pediatric audiology. We recognized a unique opportunity to contribute to socially responsible computing by applying our skills in computer graphics to improve the design and fitting process of hearing aids for infants and young children.

We chose to focus on the hearing aid as it represents a blend of technological sophistication and profound social impact—facilitating improved communication abilities for young children who are at a critical stage of language and social development. Our goal was not only to create a tool that helps visualize how a hearing aid fits on an ear but also to make this tool interactive, allowing users to understand and manipulate the hearing aid model. This interactive aspect aims to educate users on the importance and functionality of hearing aids, potentially reducing stigma and increasing awareness about hearing impairments and solutions available for young children.

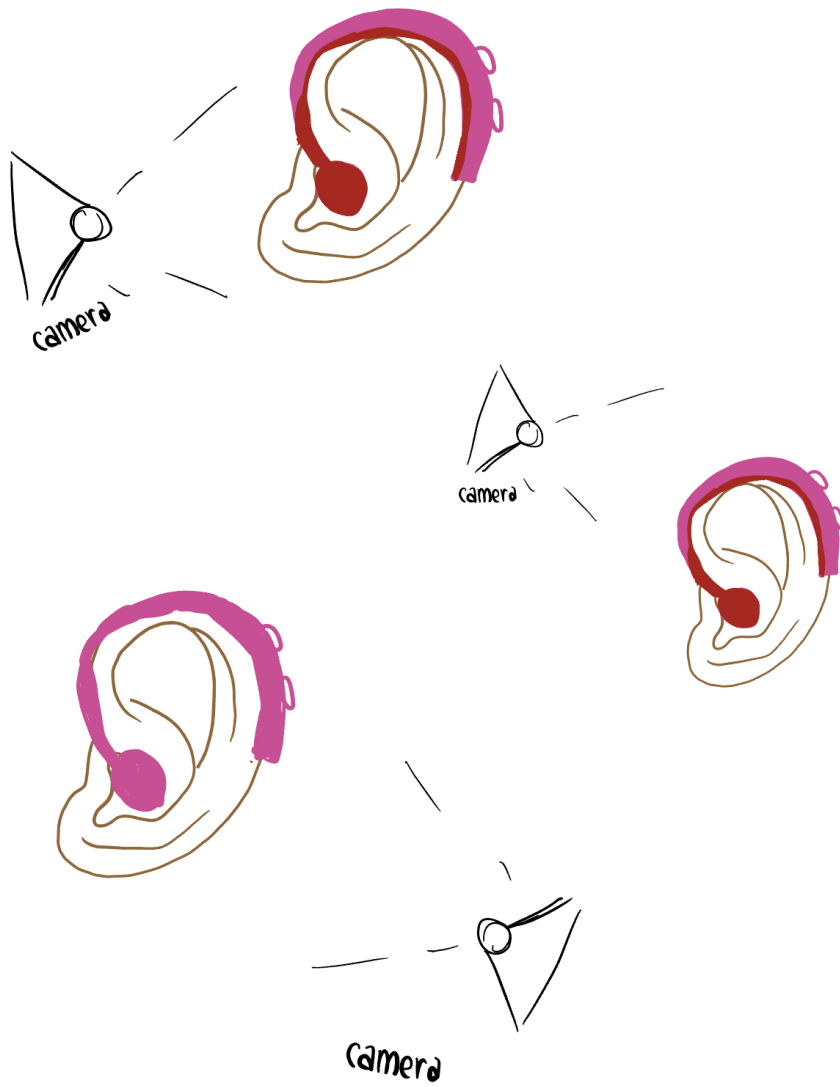
Through this project, we hope to highlight how computer graphics can be leveraged for social good, particularly in developing tools that support sensitive and essential healthcare needs.

Design

Milestone 1 Mockup



Milestone 2 Mockup

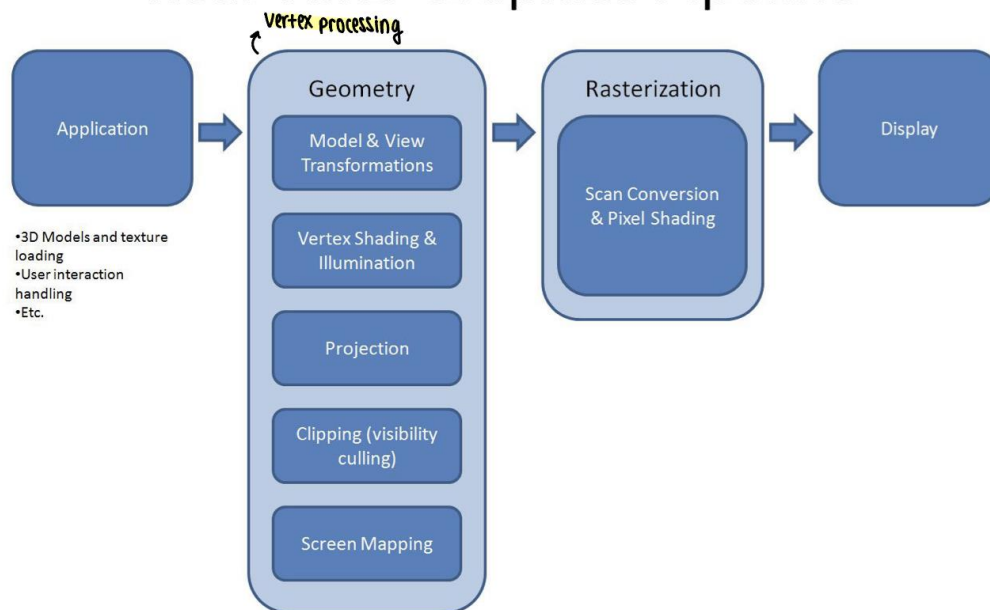


Implementation/Project Milestones

Milestone 1:

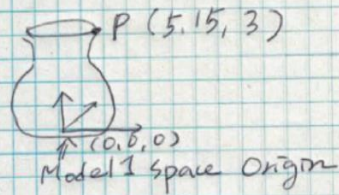
- Render Object Meshes
 - Render ear mesh
 - Don't modify
 - Render hearing aid mesh
- Establish hierarchy in hearing aid parts
- Adjust objects
 - Adjust size of ear mold
 - Adjust diameter and shape of hearing aid
- Apply Camera class
 - Move camera view with mouse
 - Move actual camera through scene
- Be able to manipulate the hearing aid
 - Rotate the hearing aid in the x,y, and z axis
 - Move the hearing aid in the x,y, and z axis

Real-Time Graphics Pipeline

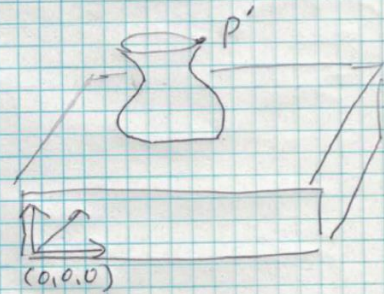


Matrix

P in Model 1 space



P' in Model 2 space



Power of matrix in Graphics

$P \rightarrow P' \rightarrow P'' \rightarrow P''' \dots$
 (Model 1) (Model 2)
 space space

$$P' = R_1 P + t_1$$

$$P'' = R_2 P' + t_2$$

$$P''' = R_3 P'' + t_3$$

$$\begin{aligned}
 P'' &= R_2 (R_1 P + t_1) \\
 &= R_2 R_1 P + R_2 t_1
 \end{aligned}$$

A better way

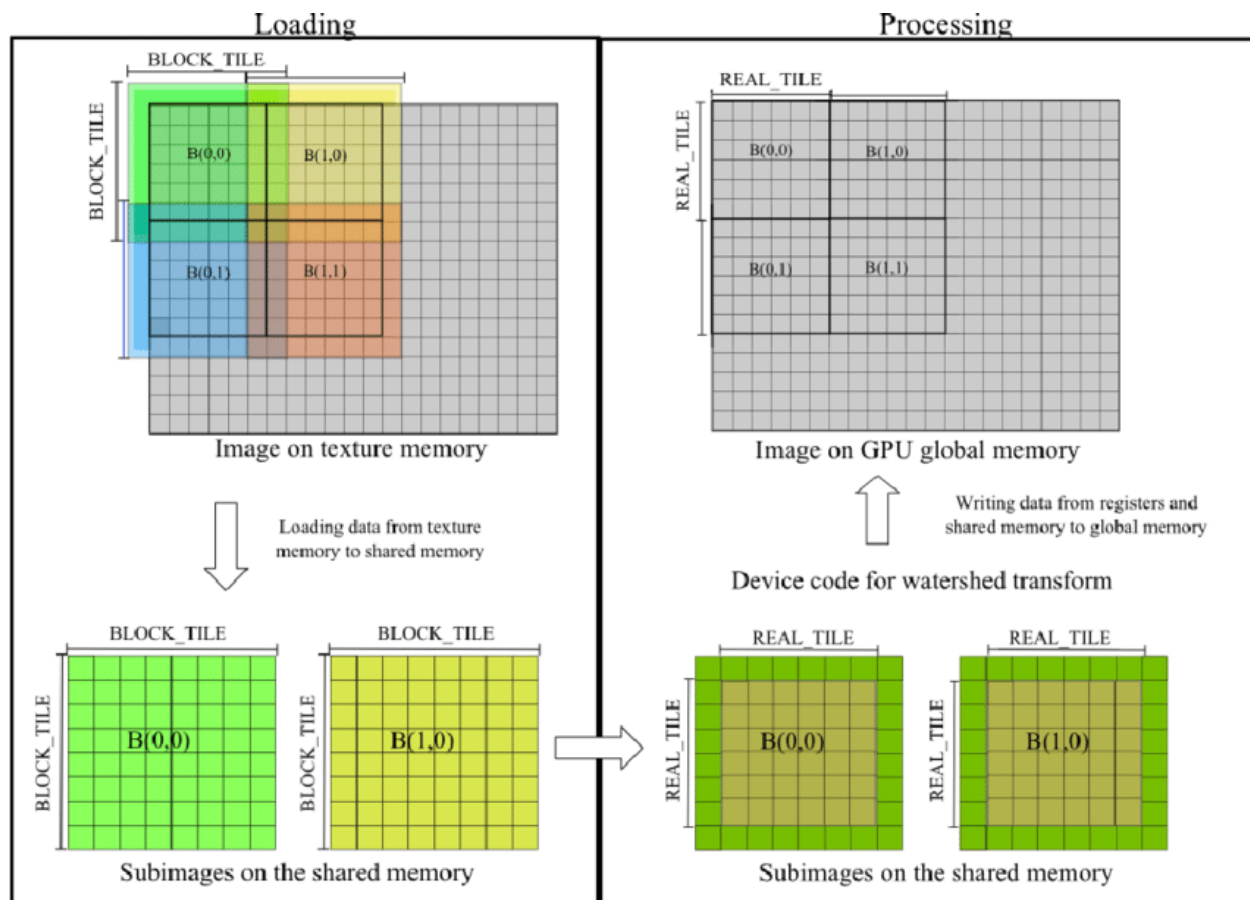
$$\begin{aligned}
 \begin{bmatrix} P' \\ 1 \end{bmatrix} &= \begin{bmatrix} R_1 & t_1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \begin{bmatrix} P \\ 1 \end{bmatrix} \\
 &= R_1 P + t_1
 \end{aligned}$$

$$\begin{aligned}
 \begin{bmatrix} P'' \\ 1 \end{bmatrix} &= \begin{bmatrix} R_2 & t_2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} P' \\ 1 \end{bmatrix} \\
 &= R_2 P' + t_2 \\
 &= \begin{bmatrix} R_2 & t_2 \end{bmatrix} \begin{bmatrix} R_1 & t_1 \end{bmatrix} \begin{bmatrix} P \\ 1 \end{bmatrix}
 \end{aligned}$$

Milestone 2:

- Create the overlapping pixel(wrong approach)
 - Define Vertex Data
 - Create vertex data for the objects that will overlap
 - Define positions, colors, texture coordinates, etc.
 - Create Vertex Buffer Objects(VBO)
 - Generate and bind VBOs for the vertex data
 - Create Vertex Array Objects(VAO)
 - Generate and bind VAOs to store vertex attribute configurations
 - Rendering Setup
 - Enable blending in OpenGL
 - Use glEnable(GL_BLEND)
 - Set blending function to control how pixels overlap
 - Use glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)
 - Drawing Objects
 - Activate the shader program
 - Use glUseProgram
 - Bind the VAO for the first object
 - Use glBindVertexArray
 - Render the first object
 - Use glDrawArrays or glDrawElements
 - Swap Buffers
 - If using double buffering swap the buffers to display the rendered scene
 - Use glfwSwapBuffers or similar function
 - Depth Testing
 - Enable depth testing
 - Use glEnable(GL_DEPTH_TEST)
 - Clear depth buffer before each frame
 - Use glClear(GL_DEPTH_BUFFER_BIT)

- Add informational text in Command Line(optional)



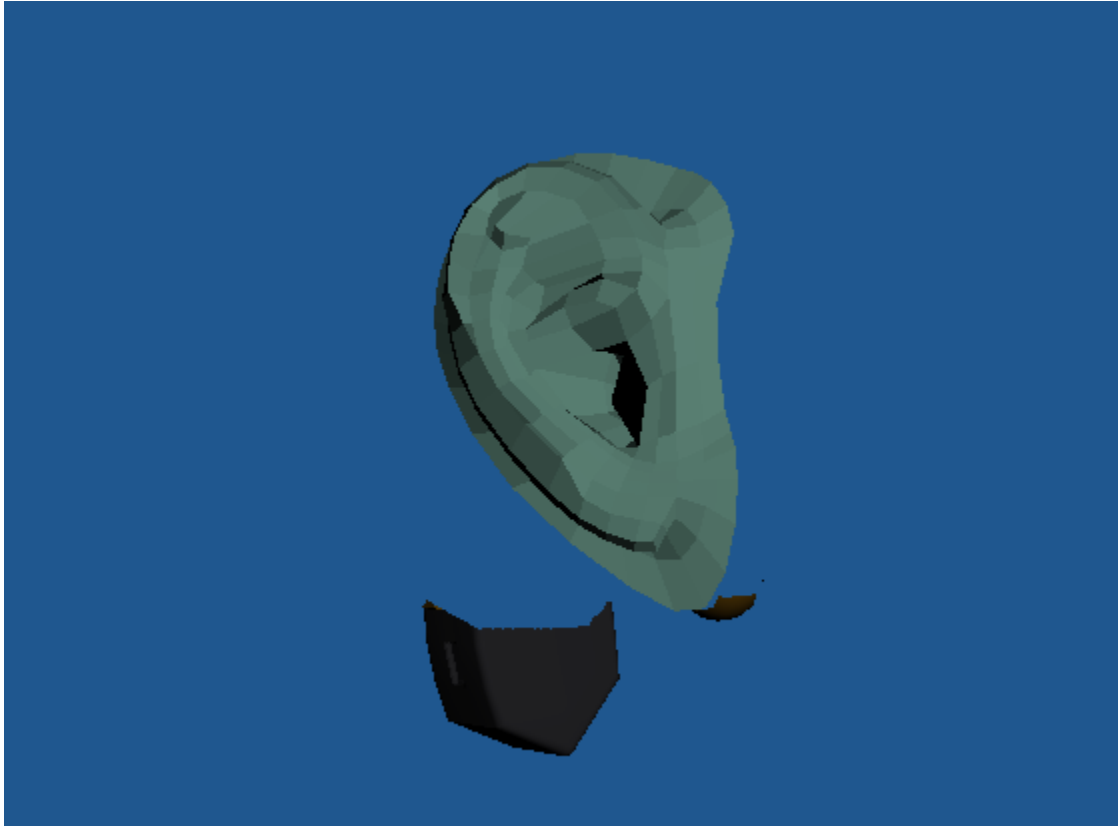
Results

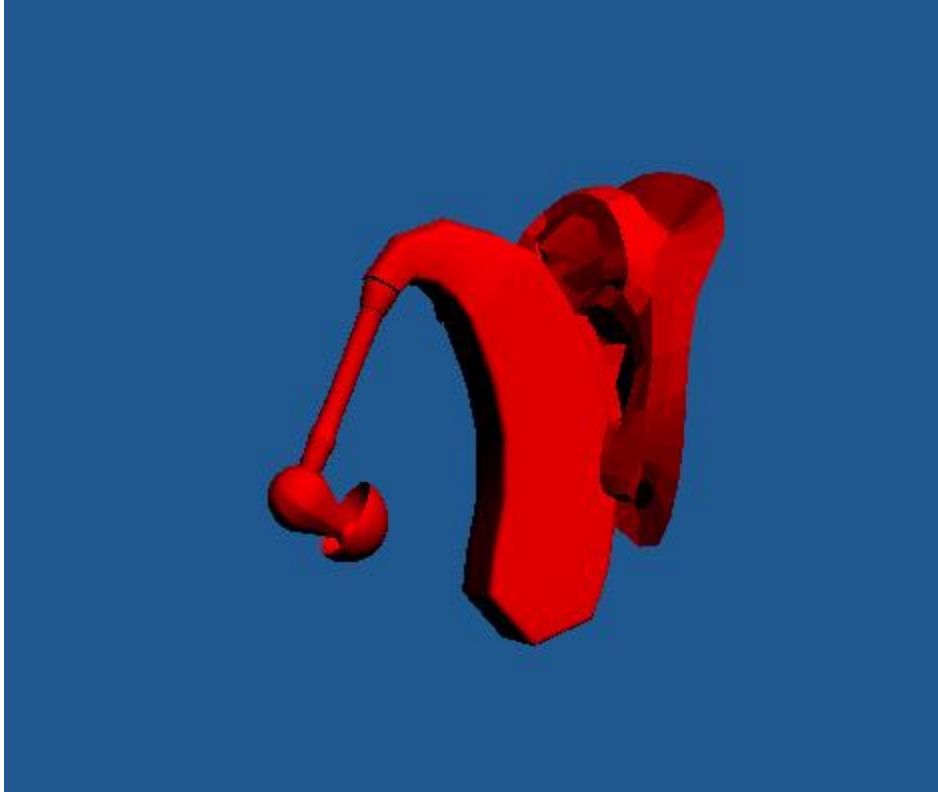
Without Failed Pixel Overlapping Implementations:





With Failed Pixel Overlapping Implementations:





Code and Resource Organization

For our code organization we decided to use GitHub. We thought it would be the easiest way to share code with each other. Also to keep track of the changes and progress we each made. We also used discord to communicate any pushes to our repository, share resources with each other and any other information.

Challenges/Problem Solving

Our biggest challenge in this project was implementing the ability to tell if an object is overlapping the other. Our approach was to use pixel overlapping which we soon found out was the wrong approach thanks to Professor Yoon. She told us that we should think of using the overlapping vertices since pixel overlapping only took account for the screen space, checking to see if the object's pixel were on the same pixel not actually

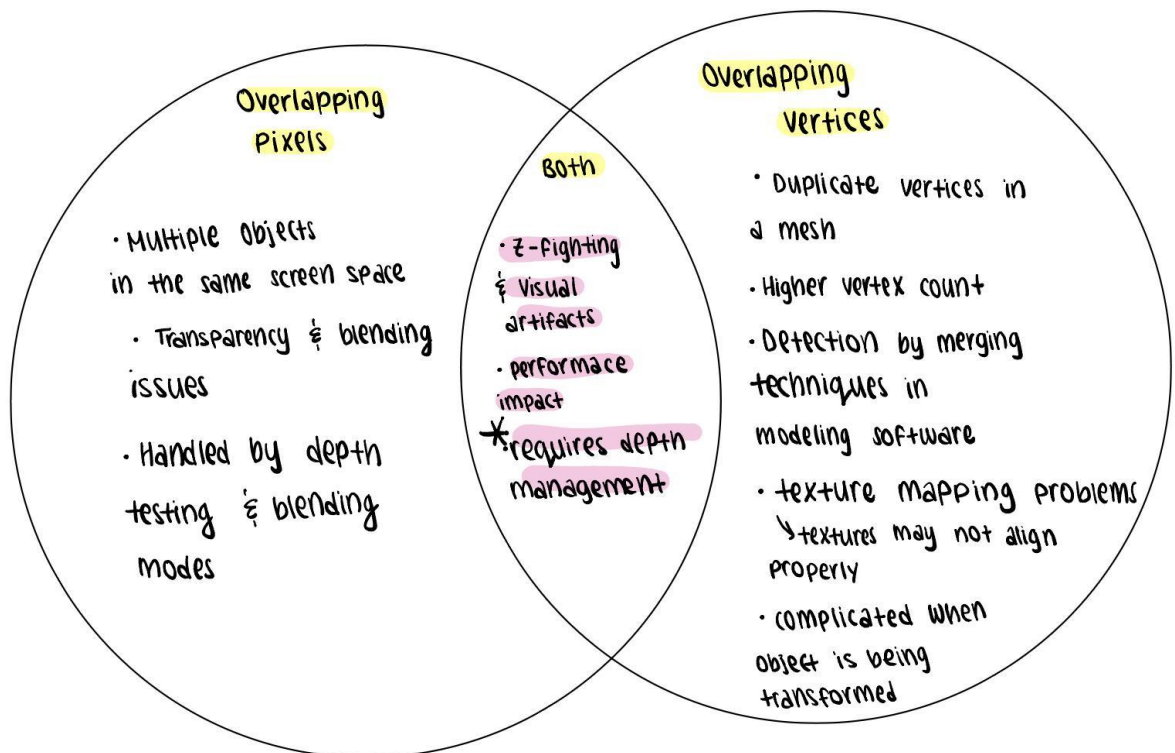
physically overlapping. However the vertex overlapping took account for the whole 3D space. Another problem we had was trying to find ways to make the rotating and movement of the hearing aid more efficient and convenient since there were a lot of keys to memorize. We decided to use the 'T' key in order to change if we were in translation mode or not, this allowed us to use the same keys for translating and rotating the object. As for the actual objects, we figured out pretty quickly that the ear mesh and hearing aid weren't made for each other. It was difficult on its own to find different meshes for ears and hearing aids. To solve this we went on blender to play around with the object files adjusting it as needed. Once modified, we also played around with the scaling of the hearing aid to help it fit better as well. A problem that isn't related to the implementation we had was using Github, more specifically the pushing. We were having trouble pushing which really ate away some time. We weren't able to fully solve this issue. Another more general problem we had was with having to re clone the project everytime after closing Visual Studio since our project wouldn't run after closing Visual Studio.

Overlapping Vertices

Meeting Notes:

Meeting 1:

- Attempting to understand the difference between overlapping vertices and overlapping pixels.



*Will have to calculate the **diameter** of both the hearing aid and of the ear

- Will be more complicated for the hearing aid because hearing aid has a unique shape
 - Potentially calculate the diameter of part of the hearing aid separately

- If calculate each piece separately → how to prevent vertex overlapping from appearing with the pieces of the hearing, since the pieces of the hearing aid are connected
 - Each piece is not being translated separately, so this should not be a problem

Meeting 2: Code Planning

- Create two arrays (containing the vertices for each object) → will be used in the boolean function
- Create an additional array (contain the vertices that overlap)
- Want to create a structure to store the vertices
 - Should contain a x, y, and z coordinates (floats)
 - Should contain normal vectors for the x, y, and z coordinates (floats)
 - Should contain texture coordinates (floats)
 - Should contain color variable, so we can set the color of the vertex as red if there is an overlap
- Simple boolean function to check if the two given vertices overlap
 - Loop through the vertices and compare
 - If overlapping return true, else false
 - Add all overlapping vertices to the array

Meeting 3: Pseudocode

Define Vertex Structure:

- Class Vertex:
 - Attributes:
 - x, y, z: float (coordinates)
 - nx, ny, nz: float (normal vectors)
 - u, v: float (texture coordinates)
 - color: tuple of three floats (RGB color)
 - Initialize Vertex Arrays for Two Objects
 - Array object1_vertices containing instances of Vertex
 - Array object2_vertices containing instances of Vertex
 - Initialize Array for Overlapping Vertices:
 - Array overlapping_vertices

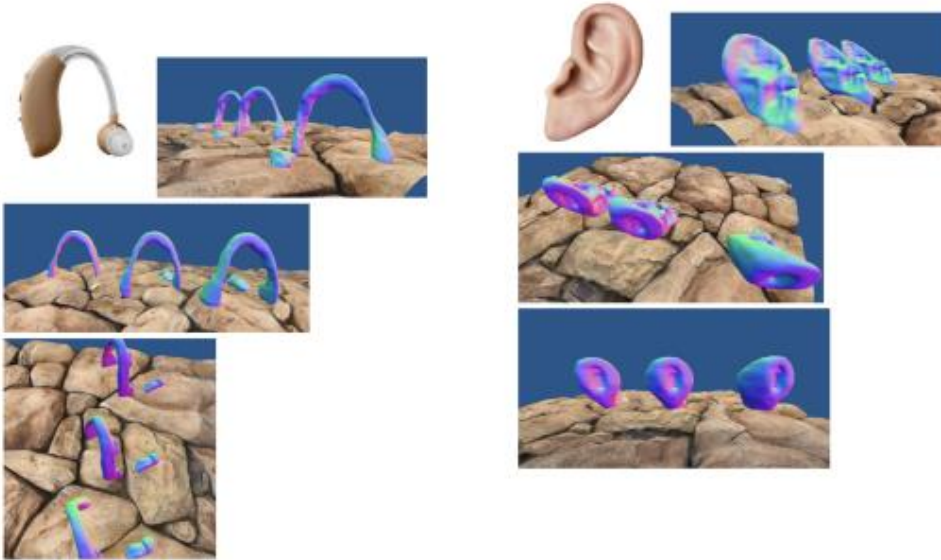
Define Function to Check Overlapping Vertices:

- Function check_overlapping_vertices(vertices1, vertices2):
 - Initialize empty array overlapping_vertices
 - For each vertex v1 in vertices1:
 - For each vertex v2 in vertices2:
 - If v1 equals v2:
 - Set v1.color to (1.0, 0.0, 0.0) (red)
 - Add v1 to overlapping_vertices
 - Return overlapping_vertices

Main Logic:

- Call check_overlapping_vertices(object1_vertices, object2_vertices)
- Store the result in overlapping_vertices
- Print or process overlapping_vertices

Collaboration with 3D Modeling Group



My Contribution

In our project, I was responsible for integrating the camera functionalities, which I accomplished by implementing `camera.c` and `camera.h`. These files are very important as they provide the necessary framework for camera operations, allowing for dynamic viewing angles and interactive user experience. My contribution improved the project's interactivity and also deepened my understanding of how camera mechanics can be effectively managed within a graphical application. This task involved a lot of research and coding to make sure we had integration and performance optimization, which was both challenging and rewarding.

Learning Reflection

Throughout this project, I believe I gain significant insights and developed my skills in several areas. Integrating camera functionalities into our application was particularly interesting, as it required a deep dive into both theoretical and practical aspects of computer graphics. I learned about the complexities of managing viewing matrices and how they interact with user inputs to manipulate the visual perspective within a 3D environment.